Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа № 3

По дисциплине: «МРЗвИС»

Тема: «Многослойный персептрон для XOR с ReLU на скрытых слоях: сравнение активационных функций»

Выполнил:

Студент 4 курса

Группы ИИ-25

Пальчастый Я.М.

Проверил:

Андренко К.В

2026

Вариант 8

Цель: а основе многослойного персептрона архитектуры 2-2-1 из ЛР №2, исследовать влияние активационной функции ReLU (Rectified Linear Unit) на скрытых слоях. Провести сравнительный анализ обучения сети с функцией активации Сигмоида (базовая конфигурация) и ReLU (новая конфигурация) по скорости сходимости, качеству обучения, форме разделяющей поверхности и устойчивости к инициализации весов.

Вариант: тот же, что и в Лабораторной работе №2

Задачи лабораторной работы

Задачи лабораторной работы

1. Взять за основу код многослойного персептрона 2-2-1 с функцией потерь BCE из ЛР №2.
2. Реализовать активационную функцию ReLU на скрытом слое и осуществить корректное обратное распространение ошибки для ReLU. На выходном слое оставить Сигмоиду.
3. Обучить сеть на полной таблице истинности в двух конфигурациях:

- **Конфигурация A (базовая):** MLP 2-2-1, Сигмоида на всех слоях, BCE, нормализованные целевые значения (0/1);
- **Конфигурация B (новая):** MLP 2-2-1, ReLU на скрытом слое, Сигмоида на выходном, BCE, нормализованные (0/1).

4. Критерий остановки для обеих конфигураций: суммарная ошибка $Es \le Ee$ (как в ЛР №2).
5. Для каждой конфигурации зафиксировать:
   - Количество эпох до критерия остановки.
   - Итоговую суммарную ошибку и accuracy.
   - Среднюю абсолютную ошибку в исходной шкале [c0; c1].
6. Провести не менее 5 запусков обучения для каждой конфигурации с разными seed и сравнить устойчивость.
7. Построить графики:
   - График сходимости (суммарная ошибка Es от эпохи) для обеих конфигураций на одних осях.
   - Диаграмма разброса числа эпох по 5 запускам (столбики) для конфигурации A и B.
8. Визуализация разделяющей поверхности:
   - Для представительного экземпляра каждой конфигурации построить карту (heatmap) выходного значения сети на плоскости $(A, B) \in [-10; 10]^2$ с наложением 4 точек обучающей выборки.
   - Описать, как ReLU изменяет форму разделяющей поверхности (кусочно-линейные границы vs гладкие).
9. Реализовать режим функционирования сети:
   - Пользователь вводит (A, B) из [−10; 10].
   - Сеть выводит: выход в нормализованной шкале, выход в исходной шкале [c0; c1], класс.

- Продемонстрировать на 4 обучающих примерах + 2–3 дополнительных парах.

10. Создать таблицу сравнения (статистика по 5 запускам, MAE для обеих конфигураций).
11. Вывод: ответить на все вопросы и провести анализ результатов.

Вариант 8 ($c0 = -8, c1 = 3$)

A	B	A ⊕ B
−8	−8	−8
−8	3	3
3	−8	3
3	3	−8

2. Код программы

```
3. import sys
   import numpy as np
   import matplotlib.pyplot as plt

   c0, c1 = -8.0, 3.0
   X = np.array([[-8, -8], [-8, 3], [3, -8], [3, 3]], dtype=float)
   Y_raw = np.array([[-8], [3], [3], [-8]], dtype=float)

   def normalize(x, lo, hi):
       return (x - lo) / (hi - lo)

   def denormalize(y, lo, hi):
       return lo + y * (hi - lo)

   Xn = normalize(X, c0, c1)
   Yn = normalize(Y_raw, c0, c1)

   EPS = 1e-12
   def sigmoid(s):
       return 1.0 / (1.0 + np.exp(-s))

   def dsigmoid(y):
       return y * (1.0 - y)

   def relu(s):
       return np.maximum(0.0, s)

   def drelu(s):
       return (s > 0).astype(float)

   class MLP221:
       def __init__(self, hidden="sigmoid", lr=0.5, seed=7):
           rng = np.random.RandomState(seed)
           self.W1 = rng.uniform(-0.1, 0.1, (2, 2)); self.T1 = rng.uniform(-0.1,
   0.1, (1, 2))
           self.W2 = rng.uniform(-0.1, 0.1, (2, 1)); self.T2 = rng.uniform(-0.1,
   0.1, (1, 1))
           self.lr, self.hidden = lr, hidden
           self.rng = np.random.RandomState(seed + 1000)

       def forward(self, x):
           x = np.asarray(x, dtype=float).reshape(1, -1)
           s1 = x @ self.W1 - self.T1
           y1 = relu(s1) if self.hidden == "relu" else sigmoid(s1)
           y2 = sigmoid(y1 @ self.W2 - self.T2)
```

```
        return s1, y1, y2

    def error(self, y, e):
        y = np.clip(y, EPS, 1 - EPS)
        return -np.sum(e * np.log(y) + (1 - e) * np.log(1 - y))

    def train_step(self, x, e):
        x, e = x.reshape(1, -1), e.reshape(1, -1)
        s1, y1, y2 = self.forward(x)
        delta2 = y2 - e
        back = delta2 @ self.W2.T
        d1 = drelu(s1) if self.hidden == "relu" else dsigmoid(y1)
        delta1 = back * d1
        self.W2 -= self.lr * (y1.T @ delta2); self.T2 += self.lr * delta2
        self.W1 -= self.lr * (x.T @ delta1);  self.T1 += self.lr * delta1
        return self.error(y2, e)

    def predict(self, x):
        return self.forward(x)[2][0, 0]

    def fit(self, X, Y, Ee, max_epochs=20000):
        hist = []
        for epoch in range(1, max_epochs + 1):
            idx = self.rng.permutation(len(X))
            Es = sum(self.train_step(X[i], Y[i]) for i in idx)
            hist.append(Es)
            if Es <= Ee:
                return epoch, Es, True, hist
        return max_epochs, Es, False, hist

def accuracy(m, Xn, Y, thr=0.5):
    return np.mean([int(m.predict(x) >= thr) == int(y[0]) for x, y in zip(Xn,
Y)])

def mae_orig(m, Xn, Yraw):
    p = np.array([denormalize(m.predict(x), c0, c1) for x in Xn])
    return float(np.mean(np.abs(p - Yraw.ravel())))

def final_Es(m, Xn, Y):
    return sum(m.error(m.forward(x)[2], y.reshape(1, -1)) for x, y in zip(Xn,
Y))

def infer(m, A, B, name):
    yn = m.predict(normalize(np.array([A, B], dtype=float), c0, c1))
    yh = denormalize(yn, c0, c1)
    cls = c0 if abs(yh - c0) <= abs(yh - c1) else c1
    print(f"[{name}] A={A:g}, B={B:g} -> y_norm={yn:.4f}, y_hat={yh:.4f}, класс
{cls:g}")

CONFIGS = {
    "A (Sigmoid)": dict(hidden="sigmoid", Ee=0.05),
    "B (ReLU)": dict(hidden="relu",    Ee=0.05),
}
SEEDS = [1, 2, 3, 4, 5]
MAX_EPOCHS = 20000

def run(cfg, seed):
    m = MLP221(hidden=cfg["hidden"], seed=seed)
    ep, Es, ok, hist = m.fit(Xn, Yn, cfg["Ee"], MAX_EPOCHS)
    return m, dict(seed=seed, epochs=ep, Es=final_Es(m, Xn, Yn), ok=ok,
                   acc=accuracy(m, Xn, Yn), mae=mae_orig(m, Xn, Y_raw),
hist=hist)

if __name__ == "__main__":
    res, models = {}, {}
    for name, cfg in CONFIGS.items():
        res[name], models[name] = [], []
        for s in SEEDS:
```

```
            m, r = run(cfg, s); res[name].append(r); models[name].append(m)
        print(f"\n Конфигурация {name}, Ee={cfg['Ee']}")
        print("seed | эпох | Es | acc | MAE[c0;c1] | достигнут Ee")
        for r in res[name]:
            print(f"{r['seed']:>4} | {r['epochs']:>5} | {r['Es']:.5f} |
{r['acc']:.2f} | {r['mae']:.4f} | {r['ok']}")

    print("\n Сводная таблица (5 запусков)")
    print(f"{'Конфигурация':<14}|{'Сошлось':>8}|{'эпохи
min':>10}|{'max':>7}|{'mean':>8}|{'std':>7}|{'Es mean':>9}|{'acc
mean':>9}|{'MAE mean':>9}|{'MAE std':>8}")
    for name in CONFIGS:
        R = res[name]; ep = [r['epochs'] for r in R]
        print(f"{name:<14}|{sum(r['ok'] for r in
R):>6}/5|{min(ep):>10}|{max(ep):>7}|{np.mean(ep):>8.0f}|{np.std(ep):>7.0f}|"
              f"{np.mean([r['Es'] for r in R]):>9.4f}|{np.mean([r['acc'] for r
in R]):>9.2f}|"
              f"{np.mean([r['mae'] for r in R]):>9.4f}|{np.std([r['mae'] for r
in R]):>8.4f}")

    rep = {}
    for name in CONFIGS:
        ok = [i for i, r in enumerate(res[name]) if r['ok']] or
list(range(len(SEEDS)))
        rep[name] = sorted(ok, key=lambda k: res[name][k]['epochs'])[len(ok) //
2]
    colors = {"A (Sigmoid)": "tab:blue", "B (ReLU)": "tab:red"}

    plt.figure(figsize=(8, 5))
    for name, cfg in CONFIGS.items():
        r = res[name][rep[name]]; h = r["hist"]
        plt.plot(range(1, len(h) + 1), h, color=colors[name], label=f"{name},
seed={r['seed']}")
    plt.axhline(0.05, color="k", ls=":", label="Ee = 0.05")
    plt.yscale("log"); plt.xscale("log")
    plt.xlabel("Эпоха"); plt.ylabel("Суммарная ошибка Es");
plt.title("Сходимость: Sigmoid vs ReLU (BCE)")
    plt.legend(); plt.grid(alpha=.3); plt.tight_layout();
plt.savefig("1_convergence.png", dpi=150); plt.show(); plt.close()

    plt.figure(figsize=(8, 5)); w = 0.38; xs = np.arange(len(SEEDS))
    for k, name in enumerate(CONFIGS):
        for j, r in enumerate(res[name]):
            plt.bar(xs[j] + (k - .5) * w, r["epochs"], w, color=colors[name],
edgecolor="k",
                    hatch=None if r["ok"] else "//", alpha=1 if r["ok"] else
.5,
                    label=name if j == 0 else None)
    plt.bar(0, 0, color="w", edgecolor="k", hatch="//", label="Не достигнут
Ee")
    plt.xticks(xs, [f"seed {s}" for s in SEEDS]); plt.yscale("log")
    plt.xlabel("Запуск"); plt.ylabel("Число эпох"); plt.title("Устойчивость
сходимости (5 запусков)")
    plt.legend(); plt.grid(axis="y", alpha=.3); plt.tight_layout();
plt.savefig("2_epochs.png", dpi=150); plt.show(); plt.close()

    g = np.linspace(-10, 10, 200); GA, GB = np.meshgrid(g, g)
    fig, axs = plt.subplots(1, 2, figsize=(12, 5))
    for ax, name in zip(axs, CONFIGS):
        m = models[name][rep[name]]
        Z = np.array([[denormalize(m.predict(normalize(np.array([a, b]), c0,
c1)), c0, c1)
                       for a in g] for b in g])
        cs = ax.contourf(GA, GB, Z, levels=30, cmap="viridis")
        ax.contour(GA, GB, Z, levels=[(c0 + c1) / 2], colors="w", linestyles="-
-")
        for xi, yi in zip(X, Y_raw.ravel()):
            ax.scatter(*xi, c=[[1, .85, 0] if yi == c1 else [1, .2, .2]],
```

```
s=140, edgecolors="k")
        ax.scatter([], [], c="gold", edgecolors="k", label=f"класс {c1:g}")
        ax.scatter([], [], c="tab:red", edgecolors="k", label=f"класс {c0:g}")
        ax.set_xlabel("A"); ax.set_ylabel("B"); ax.set_title(f"Выход сети в
[{c0:g};{c1:g}] — {name}")
        ax.legend(loc="lower right"); fig.colorbar(cs, ax=ax, label="ŷ в шкале
[c0; c1]")
    plt.tight_layout(); plt.savefig("3_surface.png", dpi=150); plt.show();
plt.close()

    plt.figure(figsize=(6, 5))
    means = [np.mean([r["mae"] for r in res[n]]) for n in CONFIGS]
    stds = [np.std([r["mae"] for r in res[n]]) for n in CONFIGS]
    bars = plt.bar(list(CONFIGS), means, yerr=stds, capsize=5, color=[colors[n]
for n in CONFIGS])
    for b, v in zip(bars, means): plt.text(b.get_x() + b.get_width()/2, v,
f"{v:.4f}", ha="center", va="bottom")
    plt.ylabel("MAE в шкале [c0; c1]"); plt.title("MAE (среднее ± std, 5
seed)")
    plt.tight_layout(); plt.savefig("4_mae.png", dpi=150); plt.show();
plt.close()

    tests = [tuple(x) for x in X] + [(0, 0), (-5, 7), (9.5, -3)]
    for name in CONFIGS:
        print(f"\n Функционирование, конфигурация {name}")
        for a, b in tests: infer(models[name][rep[name]], a, b, name)

    if sys.stdin.isatty():
        m = models["B (ReLU)"][rep["B (ReLU)"]]
        while True:
            raw = input("Введите A B из [-10;10] (пусто — выход): ").strip()
            if not raw: break
            try:
                a, b = map(float, raw.split()); assert -10 <= a <= 10 and -10
<= b <= 10
            except Exception:
                print("Нужны два числа из [-10; 10]"); continue
            infer(m, a, b, "ReLU")
```

Результат программы:

C:\Users\arosl\PycharmProjects\MRZV3\.venv1\Scripts\python.exe
C:\Users\arosl\PycharmProjects\MRZV3\main.py

Конфигурация A (Sigmoid), Ee=0.05

seed	эпох	Es	acc	MAE[c0;c1]	достигнут Ee
1	4117	0.04937	1.00	0.1348	True
2	20000	1.91039	0.75	3.6868	False
3	20000	1.90999	0.75	3.6784	False
4	872	0.04943	1.00	0.1350	True
5	943	0.04937	1.00	0.1349	True

Конфигурация B (ReLU), Ee=0.05

seed	эпох	Es	acc	MAE[c0;c1]	достигнут Ee
1	20000	1.38655	0.75	2.7500	False
2	20000	1.91183	0.75	3.6166	False
3	20000	2.77477	0.50	5.5000	False
4	147	0.04890	1.00	0.1333	True

5 | 20000 | 1.91302 | 0.75 | 3.7297 | False

Сводная таблица (5 запусков)

Конфигурация	Сошлось	эпохи min	max	mean	std	Es mean	acc mean	MAE mean	MAE std
A (Sigmoid)	3/5	872	20000	9186	8907	0.7937	0.90	1.5540	1.7380
B (ReLU)	1/5	147	20000	16029	7941	1.6070	0.75	3.1459	1.7514

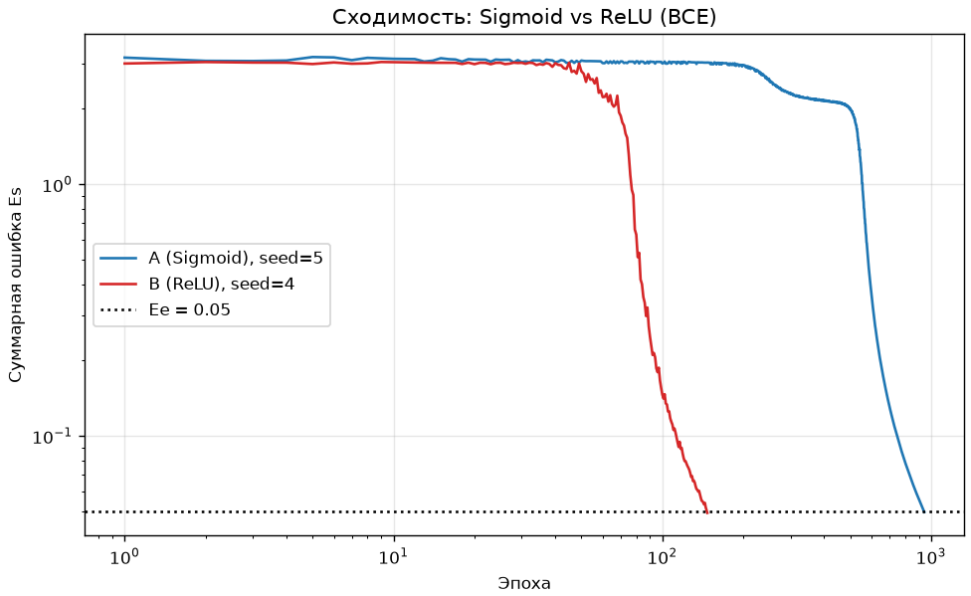
Функционирование, конфигурация A (Sigmoid)

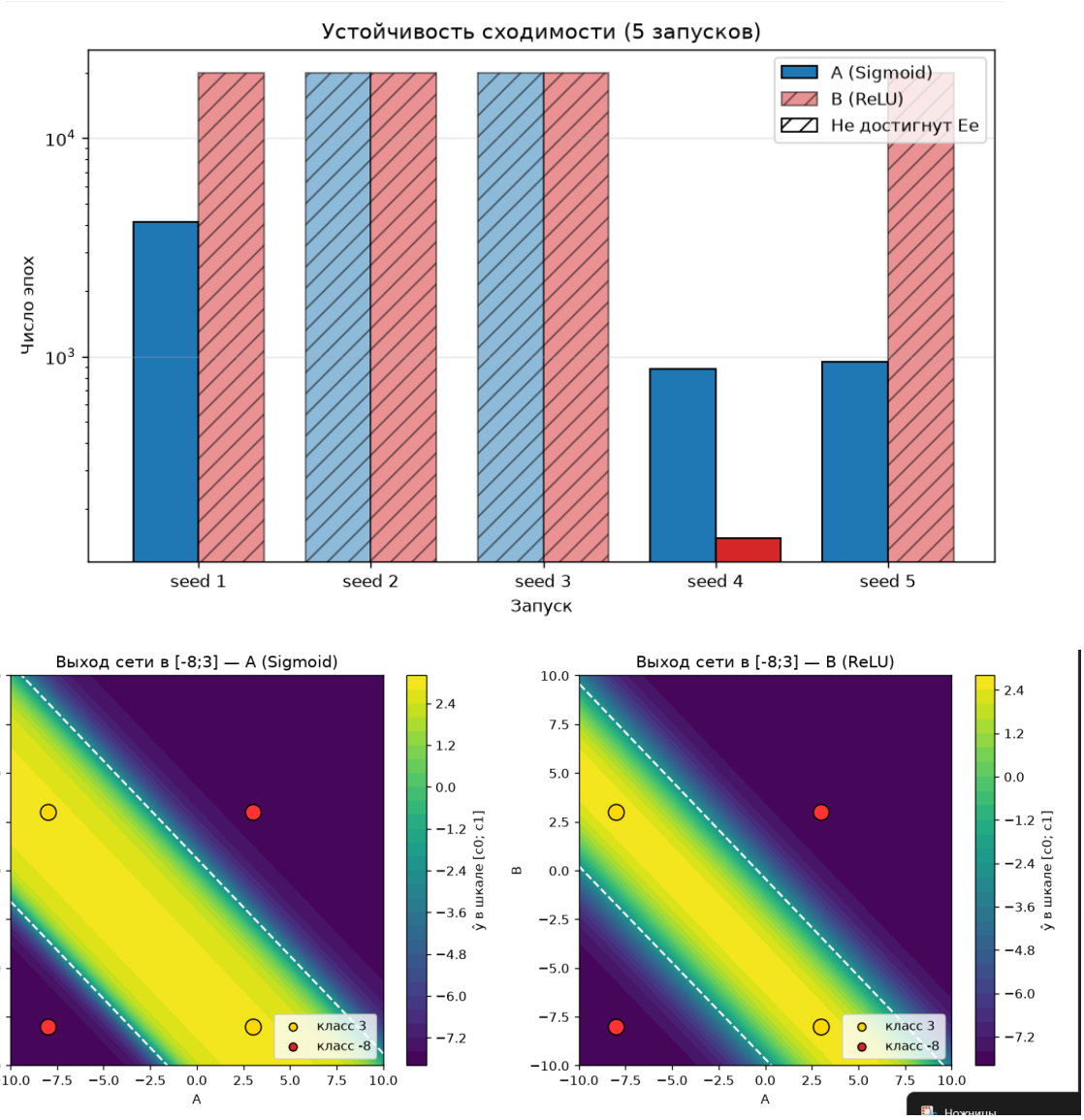
[A (Sigmoid)] A=-8, B=-8 -> y_norm=0.0092, y_hat=-7.8985, класс -8
[A (Sigmoid)] A=-8, B=3 -> y_norm=0.9890, y_hat=2.8791, класс 3
[A (Sigmoid)] A=3, B=-8 -> y_norm=0.9890, y_hat=2.8791, класс 3
[A (Sigmoid)] A=3, B=3 -> y_norm=0.0178, y_hat=-7.8038, класс -8
[A (Sigmoid)] A=0, B=0 -> y_norm=0.6687, y_hat=-0.6445, класс 3
[A (Sigmoid)] A=-5, B=7 -> y_norm=0.1825, y_hat=-5.9920, класс -8
[A (Sigmoid)] A=9.5, B=-3 -> y_norm=0.0156, y_hat=-7.8288, класс -8

Функционирование, конфигурация B (ReLU)

[B (ReLU)] A=-8, B=-8 -> y_norm=0.0052, y_hat=-7.9431, класс -8
[B (ReLU)] A=-8, B=3 -> y_norm=0.9806, y_hat=2.7870, класс 3
[B (ReLU)] A=3, B=-8 -> y_norm=0.9801, y_hat=2.7812, класс 3
[B (ReLU)] A=3, B=3 -> y_norm=0.0041, y_hat=-7.9554, класс -8
[B (ReLU)] A=0, B=0 -> y_norm=0.4105, y_hat=-3.4843, класс -8
[B (ReLU)] A=-5, B=7 -> y_norm=0.1118, y_hat=-6.7700, класс -8
[B (ReLU)] A=9.5, B=-3 -> y_norm=0.0026, y_hat=-7.9710, класс -8

Process finished with exit code 0





Ответы на вопросы:

1. Какая функция обеспечила более быструю сходимость? Более устойчива?

По скорости сходимости в удачных запусках побеждает ReLU: единственный сошедшийся запуск ReLU (seed = 4) достиг Ee = 0.05 всего за 147 эпох, тогда как лучший запуск сигмоиды (seed = 4) - за 872 эпохи. На графике сходимости видно, что красная кривая ReLU обрывается вниз примерно на порядок раньше синей кривой сигмоиды.

По устойчивости уверенно побеждает сигмоида: она сошлась в 3 из 5 запусков, тогда как ReLU - лишь в 1 из 5. В четырёх «провальных» запусках ReLU ошибка застревает на уровне Es ≈ 1.39…2.77, accuracy падает до 0.50-0.75, а сеть фактически вырождается в линейный классификатор (не решает XOR). У сигмоиды провальные запуски тоже есть (2 из 5), но их меньше, и качество «удачных» моделей заметно выше.

2. Отличие формы разделяющей поверхности. Кусочно-линейность ReLU?

Обе поверхности выглядят как плавный диагональный градиент от «класса -8» (тёмный) к «классу 3» (жёлтый), разделённый пунктирной линией уровня (c0+c1)/2 = -2.5. Ожидаемая кусочно-линейность ReLU проявляется слабо. Причины:

всего 2 скрытых нейрона - число «изломов» кусочно-линейной функции не больше числа активных нейронов;

на выходе стоит сигмоида, которая сглаживает кусочно-линейный сигнал скрытого слоя;

в выбранном «представительном» запуске ReLU (seed = 4) сеть выучила почти линейное решение, поэтому поверхность визуально близка к сигмоидной.

3. Почему ReLU предпочитают в глубоких сетях? Подтверждается ли это на XOR?

Нет насыщения при x > 0, градиент не затухает, вычисляется за одну операцию (max(0, x)), разреженная активация. В глубоких сетях это критично. На XOR-задаче преимущество НЕ подтверждается. Наоборот: из-за крошечного числа нейронов и онлайн-режима любой «мёртвый» нейрон сразу обрушивает всю сеть. ReLU здесь оказалась менее надёжной, чем сигмоида.

4. Изменилась ли способность решать XOR?

Да, изменилась в сторону ухудшения устойчивости. Архитектура 2-2-1 в принципе достаточна для XOR при любой нелинейной активации, и при удачной инициализации обе конфигурации дают 100% accuracy и MAE ≈ 0.13. Но вероятность попасть в хороший минимум у ReLU заметно ниже (1/5 против 3/5).

5. «Мёртвые нейроны» при ReLU

Наблюдаются. В провальных запусках ReLU (seeds 1, 2, 3, 5) сеть застревает на Es ≈ 1.39-2.77 при accuracy 0.50-0.75.

6. Связь активации и необходимости многослойной архитектуры

XOR линейно неразделим, поэтому однослойный персептрон его не решает - нужен скрытый слой. Но одного скрытого слоя недостаточно: активация обязательно должна быть нелинейной. Если поставить тождественную функцию f(z) = z, композиция двух линейных слоёв снова даст линейную границу, и XOR снова станет неразрешимым. И сигмоида, и ReLU обеспечивают нужную нелинейность, поэтому при удачной инициализации обе конфигурации решают XOR с 2 нейронами.

7. Что показывает визуализация разделяющей поверхности? Совпадает ли граница с ожидаемой для XOR?

Уровень выхода сети $\hat{y} \in [c0; c1] = [-8; 3]$ по всей плоскости $(A, B) \in [-10; 10]^2$. Белую пунктирную линию уровня $(c0+c1)/2 = -2.5$ - фактическую границу решения. 4 обучающие точки: два «жёлтых» класса 3 (в вершинах (-8, 3) и (3, -8)) и два «красных» класса -8 (в вершинах (-8, -8) и (3, 3)). Линия проходит по диагонали и разделяет «жёлтые» и «красные» точки правильно. Однако ожидаемая для XOR граница — это две пересекающиеся прямые $A = B$ и $A = -B$, тогда как полученная поверхность даёт одну прямую (диагональ $A + B = \text{const}$). Это означает, что сеть выучила упрощённое линейное решение, а не «истинную» XOR-структуру. Причина — крайне малый объём выборки (4 точки) и всего 2 скрытых нейрона: такого представления хватает для классификации четырёх точек, но не для воспроизведения полной геометрии XOR.

Вывод:

Базовая конфигурация с сигмоидой (A) предпочтительнее - она устойчивее, точнее и не требует дополнительной борьбы с мёртвыми нейронами. ReLU стоит выбирать на глубоких и широких сетях, где избыточность нейронов и большие объёмы данных компенсируют её недостатки.